Web GIS Laboratory 4

Getting started with GeoDjango: Django, PostGIS, the admin panel and Leaflet

Dr. Shahid Nawaz Khan
Institute of Geographical Information Systems
School of Civil and Environmental Engineering
National University of Sciences and Technology, Islamabad

MS Remote Sensing and GIS

Duration	3 hours
Builds on	Labs 2 and 3: the same districts, now in a database
Software	Python, Django, PostgreSQL with PostGIS, OSGeo4W, VS Code
Deliverable	A working GeoDjango project in your repository, and a report

Contents

Before you begin 2

1 Python and a first Django project 3

2 Install PostgreSQL and PostGIS 7

3 Connect Django to PostGIS 8

4 A model with a geometry 10

5 Load the districts 12

6 The admin panel 13

7 Leaflet, fed by Django 15

8 Ask PostGIS a spatial question 18

9 Commit and submit 20

Appendix A: Troubleshooting 23

Appendix B: Reference 23

Before you begin

What changes in this lab

Until now the browser did everything. It fetched a GeoJSON file that sat next to the page and drew it. That works for a fixed dataset, but nothing can be edited, nothing is checked, and every user downloads everything.

From this lab on, the data lives in a **spatial database**, and a **web framework** stands between the database and the browser:

PostGIS	$\longleftrightarrow$	**Django**	$\longrightarrow$	**Leaflet in the browser**
stores and queries		decides what to send		draws what it receives

GeoDjango is the part of Django that understands geometry. By the end of the lab the 160 districts from Labs 2 and 3 will be stored in PostGIS, editable through Django's admin panel, and drawn by Leaflet from data Django serves.

Learning outcomes

1. Create a Python virtual environment and install Django into it.
2. Install PostgreSQL with the PostGIS extension, and create a spatial database.
3. Install the GDAL, GEOS and PROJ libraries GeoDjango depends on, and connect Django to PostGIS.
4. Define a model with a geometry field, and read the SQL Django generates for it.
5. Load a GeoJSON file into the database.
6. Use the Django admin panel to browse, filter, search and edit spatial data.
7. Serve data from the database as GeoJSON and draw it with Leaflet.

Structure and timing

This lab is mostly installation. Installation fails in different ways on different machines, so the times are a guide. Parts 1 to 7 are the core; Part 8 is an extension.

Part	What you do	Time
1	Python and a first Django project	20 min
2	Install PostgreSQL and PostGIS	25 min
3	Connect Django to PostGIS	25 min
4	A model with a geometry	15 min
5	Load the districts	15 min
6	The admin panel	20 min
7	Leaflet, fed by Django	25 min
8	Ask PostGIS a spatial question *(extension)*	15 min
9	Commit and submit	10 min

What you need

Item	Requirement
Operating system	Windows 10 or 11
Disk space	About 2 GB for PostgreSQL, PostGIS and OSGeo4W
Rights	Administrator rights, for the two installers
Data	`districts.geojson` from Lab 3, with `pop_2023`
Internet	The installers download several hundred megabytes

Keep a record as you go

Write down every version number you see, and copy any error message in full before you try to fix it. Part of your report is a record of what went wrong on your machine and how you fixed it.

1 Python and a first Django project

1.1 Check for Python, and install it if needed

Django 6.1 needs **Python 3.12 or later**. Many machines already have a Python, sometimes an old one installed by another program, so check before installing anything.

Where to check Open PowerShell: press the Windows key, type `powershell`, and press Enter. Inside VS Code you can use its terminal instead, from **Terminal**, **New Terminal**, or `Ctrl+`` . Both are the same PowerShell. Then type:

```
python --version
py --list
```

`python --version` reports the Python that runs when you type `python`. `py --list` asks the Windows Python launcher to list every version installed on the machine; it exists only if Python was installed from `python.org`.

What the answer means Find the line that matches what PowerShell printed.

You see	What to do
`Python 3.12.x`, `3.13.x` or higher	You have what you need. Skip to section 1.2
`Python 3.11.x` or lower	Too old for Django 6.1. Install 3.13 as below. The old one can stay; the two do not conflict
`Python was not found; run without arguments to install from the Microsoft Store`	Python is not installed. That message comes from a Windows shortcut, not from Python. Install 3.13 as below
`python : The term 'python' is not recognized`	Python is not installed, or was installed without being added to `PATH`. Install 3.13 as below
Nothing at all, or the Microsoft Store opens	The same Windows shortcut. Install 3.13 as below

Download the installer

1. Go to `https://www.python.org/downloads/windows/`.
2. Under **Stable Releases**, find the newest **Python 3.13** release.
3. Under it, click **Windows installer (64-bit)**. The file is called something like `python-3.13.9-amd64.exe`; the last number changes with each release, and any 3.13 is fine.

Do not take Python from the Microsoft Store. It installs into a protected folder and behaves differently with virtual environments.

Run the installer

1. On the first screen, before anything else, tick **Add python.exe to PATH** at the bottom. This is the one step people miss, and the cause of most "not recognized" errors later.
2. Leave **Use admin privileges when installing py.exe** ticked.
3. Click **Install Now**. It installs for your user account, into `C:\Users\<you>\AppData\Local\Programs\Python\Python313`.
4. If the last screen offers **Disable path length limit**, click it. Windows otherwise limits paths to 260 characters, which some Python packages exceed.
5. Click Close.

Without the installer

If you prefer the command line, this installs the same thing:

```
winget install Python.Python.3.13
```

Check that it worked Close every PowerShell window and every VS Code window, then open a new one. A window that was already open still has the old `PATH` and will not find the new Python. Then:

```
python --version
pip --version
where.exe python
```

`python --version` should now print `Python 3.13` followed by a release number, and `pip --version` should mention `python 3.13`. `where.exe python` lists every `python.exe` on `PATH` in the order Windows searches them. The first line must be the one in `...\Programs\Python\Python313`.

If the first line is `WindowsApps\python.exe`

That is the Microsoft Store shortcut, and it is being found before your real Python. Turn it off: open Windows **Settings**, **Apps**, **Advanced app settings**, **App execution aliases**, and switch off the two entries for `python.exe` and `python3.exe`. Open a new PowerShell and check again.

If you have more than one Python

When `py --list` shows several versions and `python --version` still reports an old one, name the version explicitly when you create the virtual environment in the next section:

```
py -3.13 -m venv venv
```

Once that environment is activated, `python` inside it is 3.13, whatever the rest of the machine uses.

1.2 Create the project folder and a virtual environment

Work inside your course repository, in a new folder:

```
cd C:\webgis-lab
mkdir lab4
cd lab4
python -m venv venv
venv\Scripts\Activate.ps1
```

The prompt now starts with `(venv)`. Everything you install with `pip` from here goes into the `venv` folder, not into Python itself.

If PowerShell refuses to run Activate.ps1

A message saying that running scripts is disabled on this system means PowerShell's execution policy blocks it. Allow scripts for your own account, once, then activate again:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Why a virtual environment Each project gets its own copy of its libraries, at its own versions. A project from another course that needs a different Django cannot break this one, and `requirements.txt` records exactly what this project needs.

1.3 Install Django and the PostgreSQL driver

```
pip install django "psycopg[binary]"
pip freeze > requirements.txt
python -m django --version
```

`psycopg` is how Python talks to PostgreSQL. The `[binary]` part installs a ready-built version, so no compiler is needed.

1.4 Start a project and an app

```
django-admin startproject webgis .
python manage.py startapp districts
```

Note the dot at the end of the first line: it puts the project in the current folder instead of a new one.

File or folder	Holds
`manage.py`	The command you run everything through
`webgis/settings.py`	The project's configuration
`webgis/urls.py`	Which URL goes to which code
`districts/`	An **app**: one part of the project, here everything about districts

1.5 Run it

```
python manage.py migrate
python manage.py runserver
```

Open `http://127.0.0.1:8000/`. You should see a page saying the install worked. Then open `http://127.0.0.1:8000/admin/`: a login page. There is no user yet; that comes in Part 6.

`migrate` created a database. By default it is SQLite, a single file called `db.sqlite3`. SQLite cannot store geometry properly, so Part 3 replaces it. Stop the server with `Ctrl+C` and delete `db.sqlite3`.

Checkpoint 1

A screenshot of the "install worked" page with the address bar visible, and the output of `python -m django --version`.

---

2 Install PostgreSQL and PostGIS

2.1 PostgreSQL

From `postgresql.org`, under Download, Windows, take the installer for the latest version (17 or later). Run it, and accept the defaults except for these:

Screen	What to do
Select Components	Keep all four: the server, pgAdmin 4, Stack Builder and the command line tools
Password	The password for the `postgres` superuser. Choose one and **write it down**; it cannot be recovered
Port	Leave it at `5432`
Last screen	Leave **Launch Stack Builder** ticked, and click Finish

2.2 PostGIS, through Stack Builder

PostgreSQL on its own knows nothing about geometry. PostGIS is an extension that adds geometry types, spatial indexes and several hundred spatial functions.

1. In Stack Builder, choose your PostgreSQL installation from the list, then Next.
2. Open **Spatial Extensions** and tick the **PostGIS** bundle for your PostgreSQL version.
3. Download, install, and accept the defaults throughout, including any questions about environment variables.

2.3 Create a spatial database

Open **pgAdmin 4**, and connect to the server with your password.

1. Right-click **Databases**, **Create**, **Database**. Name it `webgis`, and Save.
2. Select `webgis`, then **Tools**, **Query Tool**.
3. Run these two statements:

```
CREATE EXTENSION postgis;
SELECT postgis_full_version();
```

The second returns one long line listing PostGIS, GEOS and PROJ with their versions. That line is your proof that the database can hold geometry.

Checkpoint 2

A screenshot of pgAdmin with the `webgis` database selected and the result of `postgis_full_version()` visible.

---

3 Connect Django to PostGIS

Three things are needed: the libraries GeoDjango uses to handle geometry in Python, the password, and the settings.

3.1 GDAL, GEOS and PROJ, through OSGeo4W

GeoDjango does not do the geometry work itself. It calls three C libraries: **GDAL** to read and write formats, **GEOS** for geometry operations, **PROJ** for coordinate systems. QGIS uses the same three. On Windows, the usual way to get them is OSGeo4W.

1. Download `osgeo4w-setup.exe` from `trac.osgeo.org/osgeo4w`.
2. Run it and choose **Express Install**, then the default download site.
3. From the list of packages, tick **GDAL** only, and continue. It installs to `C:\OSGeo4W`.

If the OSGeo4W installer will not run

QGIS for Windows is built on OSGeo4W and carries the same libraries. If you installed QGIS for Lab 2, you can point Django at its folder instead, for example `C:\Program Files\QGIS 3.40.10`. Use the exact folder name on your machine in the settings below. Use this only if OSGeo4W fails.

3.2 Store the password outside the code

Your settings file will be committed to GitHub, so the database password must not be written in it. Store it in an environment variable instead:

```
setx WEBGIS_DB_PASSWORD "the-password-you-chose"
```

`setx` saves the variable for every **new** window. Close VS Code completely and open it again, then activate the environment again with `venv\Scripts\Activate.ps1`.

3.3 Edit the settings

Open `webgis/settings.py` and make three changes.

One: tell Python where OSGeo4W is Replace the line `from pathlib import Path` near the top, and add the block below `BASE_DIR`:

webgis/settings.py, top of the file

```
import os
from pathlib import Path

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent

# GeoDjango needs the GDAL, GEOS and PROJ libraries. On Windows they come
# from OSGeo4W; putting its bin folder on PATH lets Django find them.
if os.name == "nt":
    OSGEO4W = r"C:\OSGeo4W"
    os.environ["OSGEO4W_ROOT"] = OSGEO4W
    os.environ["GDAL_DATA"] = OSGEO4W + r"\share\gdal"
    os.environ["PROJ_DATA"] = OSGEO4W + r"\share\proj"
    os.environ["PROJ_LIB"] = OSGEO4W + r"\share\proj"
    os.environ["PATH"] = OSGEO4W + r"\bin;" + os.environ["PATH"]
```

`os.name == "nt"` is true only on Windows, so the same settings file still works on a Linux server later.

Two: add GeoDjango and your app webgis/settings.py, INSTALLED_APPS

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "django.contrib.gis",
    "districts",
]
```

Three: replace SQLite with PostGIS webgis/settings.py, DATABASES

```
# The password is read from an environment variable so that it is never
# written into a file that gets committed to GitHub.
DATABASES = {
    "default": {
```

```
        "ENGINE": "django.contrib.gis.db.backends.postgis",
        "NAME": "webgis",
        "USER": "postgres",
        "PASSWORD": os.environ.get("WEBGIS_DB_PASSWORD", ""),
        "HOST": "localhost",
        "PORT": "5432",
    }
}
```

3.4 Check each piece separately

Run these one at a time. Each tests one link in the chain, so the first one to fail tells you where the problem is.

```
python manage.py shell -c "from django.contrib.gis import gdal, geos; print(gdal.gdal_version(),
     geos.geos_version())"
python manage.py check
python manage.py migrate
```

Command	Proves
The `shell` line	Python can load GDAL and GEOS. It prints their version numbers
`check`	The settings are valid
`migrate`	Django can log in to PostGIS and create its tables there

In pgAdmin, refresh `webgis`, then open **Schemas**, **public**, **Tables**. Django's own tables are now there, next to `spatial_ref_sys`, which belongs to PostGIS.

Checkpoint 3

The output of the three commands, and a screenshot of the tables in pgAdmin.

---

4 A model with a geometry

4.1 The model

A **model** is a Python class that Django turns into a database table. Each attribute becomes a column.

districts/models.py, complete

```
from django.contrib.gis.db import models


class District(models.Model):
    name = models.CharField(max_length=100)
```

```
    province = models.CharField(max_length=50)
    pcode = models.CharField(max_length=10, unique=True)
    pop_2023 = models.IntegerField(null=True, blank=True)
    area_km2 = models.FloatField()
    geom = models.MultiPolygonField(srid=4326)

    class Meta:
        ordering = ["province", "name"]

    def __str__(self):
        return f"{self.name} ({self.province})"
```

Line	Why
`from django.contrib.gis.db`	The GeoDjango version of `models`, which adds the geometry fields
`null=True, blank=True`	25 districts have no census figure. Empty must be allowed
`MultiPolygonField`	Some districts are in more than one piece
`srid=4326`	WGS 84 longitude and latitude, the same as the GeoJSON
`__str__`	How a district is named in the admin and in the shell

4.2 From model to table

```
python manage.py makemigrations districts
python manage.py sqlmigrate districts 0001
python manage.py migrate
```

`makemigrations` writes a migration file describing the change. `sqlmigrate` shows the SQL that file will run, without running it. Read it. Two lines matter:

```
"geom" geometry(MULTIPOLYGON,4326) NOT NULL
CREATE INDEX "districts_district_geom_..." ON "districts_district" USING GIST ("geom");
```

The first is a real PostGIS geometry column, constrained to multipolygons in EPSG:4326. The second is a **spatial index**. You did not ask for it; GeoDjango adds one to every geometry field, because without it every spatial query would have to examine every row.

Checkpoint 4

The output of `sqlmigrate`, and a screenshot of the `districts_district` table's columns in pgAdmin.

---

5 Load the districts

5.1 The data file

```
mkdir data
copy ..\lab3\data\districts.geojson data\districts.geojson
```

5.2 A loading script

districts/load.py, complete

```
from pathlib import Path

from django.contrib.gis.gdal import DataSource
from django.contrib.gis.geos import MultiPolygon

from .models import District

data_file = Path(__file__).resolve().parent.parent / "data" / "districts.geojson"


def run():
    layer = DataSource(data_file)[0]
    print(layer.name, ":", len(layer), "features,", layer.geom_type, "geometry")

    # Start from an empty table, so the script can be run more than once
    District.objects.all().delete()

    for feature in layer:
        geom = feature.geom.geos
        geom.srid = 4326

        # The file mixes Polygon and MultiPolygon. The model stores only
        # MultiPolygon, so wrap each single polygon in a multipolygon.
        if geom.geom_type == "Polygon":
            geom = MultiPolygon(geom, srid=4326)

        District.objects.create(
            name=feature.get("district"),
            province=feature.get("province"),
            pcode=feature.get("pcode"),
            pop_2023=feature.get("pop_2023"),
            area_km2=feature.get("area_km2"),
            geom=geom,
        )

    print(District.objects.count(), "districts saved")
```

Run it:

```
python manage.py shell -c "from districts import load; load.run()"
```

It should report 160 features and 160 districts saved. Note the geometry type it prints for the layer.

Why the Polygon check The file holds 152 `Polygon` features and 8 `MultiPolygon` features, so GDAL reports the layer's geometry type as `Unknown`. The column accepts only multipolygons, and PostGIS would reject a plain polygon. Wrapping it in a `MultiPolygon` of one part changes nothing on the map and satisfies the column. GeoDjango has a shortcut for loading files, `LayerMapping`, but it refuses a layer of mixed type outright, which is why this script reads the features one by one.

5.3 Talk to the data

Open the Django shell:

```
python manage.py shell
```

and type these one at a time. This is the **ORM**: Python that Django turns into SQL.

```
District.objects.count()
District.objects.filter(province="Sindh").count()
District.objects.filter(pop_2023__isnull=True).count()
lahore = District.objects.get(name="Lahore")
lahore.pop_2023
lahore.geom.geom_type
lahore.geom.num_coords
exit()
```

The shell imports your models automatically when it starts, which is why `District` needs no import.

Checkpoint 5

The output of the loading script and of the seven shell lines.

---

6 The admin panel

Django builds a complete editing interface for your models: login, lists, search, filters and forms. You only describe what you want to see.

6.1 Create an administrator

```
python manage.py createsuperuser
```

Choose a username, an email address and a password. This account is for the admin panel only; it is not your PostgreSQL password.

6.2 Register the model

districts/admin.py, complete

```
from django.contrib.gis import admin

from .models import District


@admin.register(District)
class DistrictAdmin(admin.GISModelAdmin):
    list_display = ["name", "province", "pop_2023", "area_km2"]
    list_filter = ["province"]
    search_fields = ["name", "pcode"]
    ordering = ["province", "name"]
```

Option	Gives you
`GISModelAdmin`	A map for the geometry field, instead of a box of text
`list_display`	The columns of the list page
`list_filter`	A filter panel on the right, one entry per province
`search_fields`	A search box that matches name or P-code

6.3 Explore it

Run the server and log in at `http://127.0.0.1:8000/admin/`.

1. The home page lists **Users** and **Groups**, which Django provides, and **Districts**, which you registered.
2. Open Districts. Filter by Sindh: the count above the table reads 29. Search for `karachi`.
3. Click a column header to sort. Sort by population.
4. Open Lahore. The geometry appears on an OpenStreetMap basemap, drawn by OpenLayers. Zoom in on it.
5. Change Lahore's population to any other number and Save. In pgAdmin, run the query below and confirm that the database has changed. Then set the figure back to 13004135.
6. Click **History** on Lahore's page to see who changed what.

```
SELECT name, pop_2023 FROM districts_district WHERE name = 'Lahore';
```

What just happened The admin did not edit a copy. Every save is an `UPDATE` on the PostGIS table, and the change history is itself stored in a table that `migrate` created. Editing the geometry on the map works

the same way: the shape is sent back as geometry and written to the `geom` column.

The admin is for people you trust

Anyone who can log in can change or delete every row. It is a tool for the people who maintain the data, not a page for the public. In **Users** you can create accounts with fewer permissions, for example one that can change districts but not delete them.

Checkpoint 6

Three screenshots: the district list filtered to one province; Lahore's page with its map; and the pgAdmin query showing your changed population.

---

7 Leaflet, fed by Django

In Labs 2 and 3, Leaflet fetched `data/districts.geojson`, a file. Now it fetches a URL, and Django builds the answer from the database each time it is asked.

7.1 Two views

A **view** is a Python function that receives a request and returns a response.

districts/views.py

```
from django.core.serializers import serialize
from django.http import HttpResponse
from django.shortcuts import render

from .models import District


def map_page(request):
    return render(request, "districts/map.html")


def districts_geojson(request):
    districts = District.objects.all()

    # /api/districts/?province=Sindh returns one province only
    province = request.GET.get("province")
    if province:
        districts = districts.filter(province=province)

    data = serialize(
        "geojson",
        districts,
```

```
        geometry_field="geom",
        fields=["name", "province", "pop_2023", "area_km2"],
    )
    return HttpResponse(data, content_type="application/geo+json")
```

`serialize("geojson", ...)` turns rows into a GeoJSON FeatureCollection: the geometry field becomes each feature's `geometry`, the listed fields become its `properties`.

7.2 Two URLs

Create a new file in the app:

districts/urls.py, new file

```
from django.urls import path

from . import views

urlpatterns = [
    path("", views.map_page, name="map"),
    path("api/districts/", views.districts_geojson, name="districts-geojson"),
]
```

and include it from the project:

webgis/urls.py

```
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("districts.urls")),
]
```

Run the server and open `http://127.0.0.1:8000/api/districts/`. The browser shows GeoJSON, built from PostGIS. Then try `http://127.0.0.1:8000/api/districts/?province=Sindh` and compare the number of features.

7.3 The map page

Django looks for templates in a `templates` folder inside each app. Create the folders `districts/templates/districts/`, and in the innermost one the file `map.html`.

districts/templates/districts/map.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Districts from PostGIS</title>
  <link rel="stylesheet"
        href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css">
  <style>
    body { font-family: system-ui, sans-serif; margin: 0;
           color: #23373b; background: #f4f6f6; }
    header { padding: 16px 24px 12px; }
    h1 { margin: 0; font-size: 22px; }
    .subtitle { margin: 4px 0 0; color: #5b6a6e; }
    #map { height: calc(100vh - 90px); min-height: 400px; }
  </style>
</head>
<body>
  <header>
    <h1>Districts from PostGIS</h1>
    <p class="subtitle">
      <span id="count">loading...</span>, served by Django from the database.
    </p>
  </header>

  <div id="map"></div>

  <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
  <script>
    const DISTRICTS_URL = "{% url 'districts-geojson' %}";

    const map = L.map('map', {
      maxBounds: [[21.5, 58.5], [39.0, 80.5]],
      maxBoundsViscosity: 1.0,
      minZoom: 5
    }).setView([30.3753, 69.3451], 5);

    L.tileLayer('https://tile.openstreetmap.org/{z}/{x}/{y}.png', {
      maxZoom: 19,
      attribution: '&copy; OpenStreetMap contributors'
    }).addTo(map);

    const colours = {
      'Punjab': '#1b9e77', 'Sindh': '#d95f02', 'Khyber Pakhtunkhwa': '#7570b3',
      'Balochistan': '#e7298a', 'Islamabad': '#66a61e',
      'Azad Kashmir': '#e6ab02', 'Gilgit Baltistan': '#a6761d'
    };

    function show(value) {
      return value === null || value === undefined
        ? 'not recorded' : value.toLocaleString();
    }

    async function loadDistricts() {
      const response = await fetch(DISTRICTS_URL);
      if (!response.ok) {
        console.error('Request failed with status', response.status);
        return;
      }
      const data = await response.json();
```

```
      const layer = L.geoJSON(data, {
        style: function (feature) {
          return {
            color: '#ffffff',
            weight: 1,
            fillColor: colours[feature.properties.province] || '#888888',
            fillOpacity: 0.7
          };
        },
        onEachFeature: function (feature, featureLayer) {
          const p = feature.properties;
          featureLayer.bindTooltip(p.name + ', ' + p.province, { sticky: true });
        }
      }).addTo(map);

      map.fitBounds(layer.getBounds());
      document.getElementById('count').textContent =
        data.features.length + ' districts';
    }

    loadDdistricts();
  </script>
</body>
</html>
```

Open `http://127.0.0.1:8000/`. Hover over the districts.

`{% url 'districts-geojson' %}` Django runs the template before the browser sees it. This tag is replaced with the URL that `urls.py` named `districts-geojson`, so the page reads `const DISTRICTS_URL = "/api/districts/";`. Press `Ctrl+U` in the browser to see the page as it arrived. If you ever move the API to another address, only `urls.py` changes.

Everything you learned still applies The JavaScript is the same Leaflet as Labs 2 and 3. Only the source of the data changed. Change Lahore's province colour in the admin by editing its province name, reload the map, and see the map follow the database. Put it back afterwards.

Checkpoint 7

A screenshot of the map at `127.0.0.1:8000` with a tooltip showing, and one of the Network tab with the `api/districts/` request selected and its size visible.

---

8 Ask PostGIS a spatial question

Extension exercise

So far the database has only stored and returned rows. Its real strength is answering spatial questions. Here the browser sends a point and PostGIS says which district contains it.

8.1 The view

Add to `districts/views.py`, extending the imports at the top:

districts/views.py, added

```
from django.contrib.gis.geos import Point
from django.http import JsonResponse


def district_at(request):
    try:
        lat = float(request.GET["lat"])
        lng = float(request.GET["lng"])
    except (KeyError, ValueError):
        return JsonResponse({"error": "give lat and lng as numbers"}, status=400)

    point = Point(lng, lat, srid=4326)
    district = District.objects.filter(geom__contains=point).first()

    if district is None:
        return JsonResponse({"district": None})
    return JsonResponse({
        "district": district.name,
        "province": district.province,
        "pop_2023": district.pop_2023,
    })
```

Add its URL to `districts/urls.py`:

```
    path("api/where/", views.district_at, name="district-at"),
```

Test it directly: `http://127.0.0.1:8000/api/where/?lat=33.6423&lng=72.9906` should name Islamabad.

`Point(lng, lat)` GEOS, like GeoJSON, puts longitude first. Leaflet, as always, puts latitude first. The view takes them by name so the order cannot be confused in the URL.

`geom__contains` This becomes a PostGIS `ST_Contains` in SQL, and it uses the spatial index from Part 4 to skip every district whose bounding box does not include the point.

8.2 Call it from the map

In `map.html`, below `const DISTRICTS_URL`, add `const WHERE_URL = "{% url 'district-at' %}";` and, after `loadDistricts();`, this handler:

```
    map.on('click', async function (e) {
      const url = WHERE_URL + '?lat=' + e.latlng.lat + '&lng=' + e.latlng.lng;
      const answer = await (await fetch(url)).json();
      const text = answer.district
        ? '<strong>' + answer.district + '</strong>, ' + answer.province +
          '<br>Population: ' + show(answer.pop_2023)
```

```
        : 'Not inside any district';
      L.popup().setLatLng(e.latlng).setContent(text).openOn(map);
    });
```

8.3 The same question in SQL

In pgAdmin, run the same query by hand, then compare areas:

```
SELECT name, province FROM districts_district
WHERE ST_Contains(geom, ST_SetSRID(ST_MakePoint(72.9906, 33.6423), 4326));

SELECT name,
       round(area_km2) AS stored,
       round((ST_Area(geom::geography) / 1e6)::numeric) AS geodesic,
       round((ST_Area(ST_Transform(geom, 3857)) / 1e6)::numeric) AS web_mercator
FROM districts_district
WHERE name IN ('Lahore', 'Gilgit', 'Gwadar');
```

The second query is Part 9 of Lab 2 done in one statement. Compare the `web_mercator` column with the others and relate the ratio to each district's latitude.

Checkpoint 8

A screenshot of a popup produced by clicking the map, and the result of the area query.

---

9 Commit and submit

9.1 What not to commit

Create `lab4/.gitignore`:

lab4/.gitignore

```
venv/
__pycache__/
*.pyc
db.sqlite3
```

The `venv` folder is hundreds of megabytes and can be rebuilt at any time from `requirements.txt`. Check with `git status` that it is not listed before you commit.

9.2 Commit and push

```
cd C:\webgis-lab
```

```
git status
git add -A
git commit -m "Lab 4: GeoDjango with PostGIS, admin and Leaflet"
git push
```

This lab has no live URL

GitHub Pages serves files; it cannot run Python or a database. This project runs only on your own machine for now. Putting it on a real server is a later lab. Your submission is the repository and the report.

9.3 Write the report

Create `lab4/report.md` containing:

1. Your checkpoint screenshots.
2. The version numbers of Python, Django, PostgreSQL, PostGIS and GDAL on your machine.
3. Every installation error you met, copied in full, and what fixed it. If nothing went wrong, say so.
4. The area table from Part 8, if you completed it.
5. Your answers to the questions below.

9.4 Questions

1. In Labs 2 and 3, every visitor downloaded the whole GeoJSON file. What can a page backed by Django and PostGIS do instead? Give two examples from this lab.
2. Why does `settings.py` read the database password from an environment variable, instead of containing the password itself?
3. In Part 4, the SQL ends with an index created `USING GIST`. What does it do, and which query in this lab benefits from it?
4. The loading script wraps some geometries in a `MultiPolygon`. What would happen without that step, and why?
5. Who should have an admin login for a district dataset like this, and who should not?
6. Leaflet takes `[lat, lng]` and `Point` takes `(lng, lat)`. Where in this lab is each order used, and how does the extension's URL avoid confusing them?

9.5 Marking

Item	Marks
Django project created and running	10
PostgreSQL and PostGIS installed, `webgis` database ready	15
Django connected to PostGIS, migrations applied	15
District model, loaded with 160 districts	15
Admin registered and explored, with the database edit	10
Leaflet map drawn from the Django API	15
Report: versions, errors and answers to the six questions	20
Extension in Part 8	+10 bonus

Submit your repository link on LMS.

Appendix A: Troubleshooting

Message or symptom	Cause and fix
`python` is not recognised	Python was installed without **Add python.exe to PATH**. Run the installer again, choose Modify, and tick it. See section 1.1
`Python was not found`, or the Store opens	The Windows Store shortcut. Turn off the App execution aliases for Python, as in section 1.1
Running scripts is disabled on this system	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, then activate again
`No module named django`	The virtual environment is not active. The prompt must start with `(venv)`
Could not find the GDAL library	OSGeo4W is not in `C:\OSGeo4W`, or GDAL was not ticked in its installer. Check that `C:\OSGeo4W\bin` contains a file named `gdal` followed by digits and `.dll`
Could not find the GEOS library	As above: `geos_c.dll` must be in the same `bin` folder
`fe_sendauth: no password supplied`	`WEBGIS_DB_PASSWORD` is not set in this window. Run `setx` again if needed, then close and reopen VS Code
`password authentication failed for user "postgres"`	The variable holds the wrong password. Run `setx` with the right one and reopen VS Code
`connection refused` on port 5432	The PostgreSQL service is not running. Start it from Windows **Services**
`type "geometry" does not exist`	`CREATE EXTENSION postgis;` was not run in the `webgis` database
`database "webgis" does not exist`	Create it in pgAdmin, with exactly that name
`TemplateDoesNotExist: districts/map.html`	The file must be at `districts/templates/districts/map.html`, and `districts` must be in `INSTALLED_APPS`
`NoReverseMatch` for `districts-geojson`	The `name=` in `urls.py` and the name in `{% url %}` differ
Map page loads, no districts	Open `/api/districts/` directly. An error there is a Python error; an empty collection means the loading script was not run

Appendix B: Reference

Commands

```
venv\Scripts\Activate.ps1          activate the environment
pip install -r requirements.txt    install what the project needs
python manage.py runserver         start the development server
python manage.py makemigrations    record a change to the models
python manage.py migrate           apply it to the database
python manage.py sqlmigrate APP N  show the SQL of a migration
python manage.py shell             a Python prompt with Django loaded
python manage.py createsuperuser   an admin account
```

The pieces and where they live

File	Role
`webgis/settings.py`	Database, installed apps, OSGeo4W paths
`webgis/urls.py`	Sends `admin/` to the admin, everything else to the app
`districts/models.py`	The table
`districts/load.py`	Fills it from GeoJSON
`districts/admin.py`	How the admin shows it
`districts/views.py`	What each URL returns
`districts/urls.py`	Which URL calls which view
`districts/templates/`	The HTML Django sends to the browser

Optional, if you finish everything

1. Add a province drop-down to the map page that calls `/api/districts/?province=...`, so the filtering happens in the database rather than in the browser.
2. Add a `Facility` model with a `PointField`, register it in the admin, and add three facilities by clicking on the admin map.
3. In the admin, create a second user who can change districts but not delete them, and log in as that user to check.
4. Show the 25 districts without census data in grey, using the `pop_2023` property the API already sends.